NeuroKinematics — Studio Trajectory Layer

Görev

NeuroKinematics projesinin Studio fazına, doğrulanmış IK çözümlerini yalnızca statik eklem konfigürasyonları olarak göstermek yerine robotun mevcut konumundan hedef konuma zaman içinde hareket etmesini sağlayacak bir **Joint-Space Trajectory Generation & Motion Execution Layer** ekle.
Bu aşama mevcut IK, FK, Hybrid Solver veya Core mimarisini yeniden tasarlamamalıdır.
Amaç:
current joint state → target joint state → continuous trajectory → simulation playback hattını oluşturmak ve mevcut NeuroKinematics mimarisine temiz şekilde entegre etmektir.

---

1. Temel Tasarım İlkesi

NeuroKinematics'in IK solver'ı şu çıktıyı üretmektedir:

q_target = [q1, q2, ..., qN]

Yeni trajectory katmanı ise şu problemi çözmelidir:

q_current → q_target

ve zaman parametreli:

q(t), dq(t), ddq(t)

trajektorisini üretmelidir.

Trajectory modülü IK çözmemeli ve IK solver kodunu değiştirmemelidir.

Mimari ayrım korunmalıdır:

Target Pose ↓ IK Solver ↓ Target Joint Configuration ↓ Trajectory Generator ↓ Joint

Trajectory ↓ FK Validation ↓ 3D Simulation / Playback

---

2. Kapsam

İlk sürüm yalnızca **joint-space point-to-point trajectory generation** içermelidir.
Desteklenmesi gereken temel senaryolar:

1. Robotun mevcut konumundan tek bir hedef konfigürasyona gitmesi.
2. Robotun mevcut konumundan A hedefine, ardından B hedefine gitmesi.
3. Birden fazla waypoint'in sırayla takip edilmesi.
4. Hareket sırasında joint position, velocity ve acceleration değerlerinin hesaplanması.
5. Üretilen trajectory'nin FK kullanılarak Cartesian uzayda doğrulanması.

6. Trajectory'nin Studio/3B simülasyon tarafından frame-by-frame oynatılabilmesi.

Bu aşamada zorunlu olmayan özellikler:

- obstacle avoidance
- RRT / RRT*
- PRM
- MoveIt motion planning
- Cartesian path planning
- torque control
- dynamics simulation
- fiziksel motor kontrolü

Bunlar sonraki genişletmeler için ayrı tutulmalıdır.

---

3. Trajectory Representation

Standart bir trajectory veri yapısı oluştur.

Önerilen yapı:

Trajectory

- timestamps
- joint_positions
- joint_velocities
- joint_accelerations
- metadata

Örneğin:

timestamps.shape = [T]

joint_positions.shape = [T, DOF]

joint_velocities.shape = [T, DOF]

joint_accelerations.shape = [T, DOF]

Metadata içinde en az:

- robot_name
- start_configuration
- target_configuration
- duration
- interpolation_method

- timestep
- joint_limits
- generation_timestamp

tutulmalıdır.

---

4. Interpolation

İlk implementasyonda en az iki yöntem destekle:

4.1 Linear Joint Interpolation

q(t) = q0 + s(t)(qf - q0)
Bu yöntem baseline görevi görmelidir.
Ancak velocity discontinuity nedeniyle varsayılan yöntem olarak kullanılmamalıdır.

4.2 Quintic Polynomial Trajectory

Varsayılan yöntem quintic polynomial olmalıdır.
Her joint için:
q(0) = q0
q(T) = qf
dq(0) = 0
dq(T) = 0
ddq(0) = 0
ddq(T) = 0
sınır koşulları uygulanmalıdır.
Amaç başlangıç ve bitişte düzgün:
position
velocity
acceleration
profilleri oluşturmaktır.

---

5. Joint Limit Validation

Trajectory boyunca her timestep için:
$q_{min} \le q(t) \le q_{max}$
kontrol edilmelidir.
Joint limit ihlali varsa trajectory başarısız kabul edilmelidir.
Ayrıca mevcut robot manifesti/URDF üzerinden mümkünse:
velocity limit
acceleration limit
de okunmalı veya merkezi robot modelinden alınmalıdır.
Kontrol:
$|dq(t)| \le dq_{max}$
$|ddq(t)| \le ddq_{max}$
şeklinde yapılmalıdır.

---

6. Automatic Duration Scaling

Kullanıcı duration vermediğinde sistem güvenli bir hareket süresi hesaplayabilmelidir.
Örneğin tüm joint'ler için gereken süre hesaplanıp:
T = max(T_joint_i)
seçilmelidir.
Velocity veya acceleration limitleri aşılıyorsa trajectory duration otomatik artırılmalıdır.
Bu işlem deterministik olmalıdır.

---

7. Multi-Waypoint Support

Trajectory generator yalnızca:
q0 → q1
değil:
q0 → q1 → q2 → ... → qN
hareketlerini de desteklemelidir.

Örneğin:
Current Pose ↓ Point A ↓ Point B
senaryosu doğrudan çalışmalıdır.
API tasarımı örneği:
generate_trajectory(current_q, waypoints=[q_A, q_B], ...)
Segmentler arasında mümkün olduğunca süreklilik korunmalıdır.
İlk implementasyonda her segmentin başlangıç/bitiş velocity değerlerini sıfırlamak kabul edilebilir.
Daha ileri velocity blending sonraki aşamaya bırakılabilir.

---

8. IK Integration

Trajectory modülü hedef noktaları kendisi

çözmemelidir.
Örneğin:
Pose A Pose B
verildiğinde:
q_A = solver.solve(Pose_A)
q_B = solver.solve(Pose_B)
çalıştırılmalı, ardından:
trajectory.generate(current_q, [q_A, q_B])
kullanılmalıdır.
IK solver bağımsız kalmalıdır.

---

9. FK Validation

Trajectory'nin her timestep'inde mevcut doğrulanmış FK pipeline'ı kullanılmalıdır.
Her q(t) için:
T_ee(t) = FK(q(t))

hesaplanmalıdır.
Trajectory'nin son noktasında:
position error
orientation error
hesaplanmalı ve solver hedefiyle
karşılaştırılmalıdır.

---

10. Simulation Playback Interface

Trajectory çıktısının Studio/visualizer tarafından kullanılabilmesi için basit bir playback API oluştur.
Örneğin:
TrajectoryPlayer
fonksiyonları:
load_trajectory()
play()

pause()
stop()
seek(t)
set_speed(multiplier)
current_state()
İlk implementasyonda gerçek GUI zorunlu değildir.
Ancak arayüz daha sonra PyQt/PyVista Studio tarafından kullanılabilecek şekilde bağımsız tasarlanmalıdır.

---

11. Export

Trajectory en az aşağıdaki formatlardan biriyle dışa aktarılabilmelidir:
JSON
ve tercihen:

CSV veya NPZ
Her örnekte:
time
q1...qN
dq1...dqN
ddq1...dqN
bulunmalıdır.
ROS2 entegrasyonu için gelecekte sensor_msgs / trajectory_msgs formatına dönüştürme imkânı düşünülmeli ancak ROS2 dependency bu aşamada zorunlu olmamalıdır.

---

12. Mimari

Önerilen modüler yapı:
src/neurokinematics/ trajectory/ init.py base.py linear.py quintic.py generator.py validation.py

trajectory.py playback.py
İsimleri mevcut repo mimarisine göre adapte et.
Projede mevcut stil, typing, dataclass, config ve logging standartları varsa aynen kullan.
Duplicate robot model veya joint-limit tanımları oluşturma.
Single source of truth prensibini koru.

---

13. Testler

Ayrıntılı unit ve integration testleri oluştur.

T-S01 — Endpoint correctness

Trajectory'nin:
$q(0) \approx q_{start}$
$q(T) \approx q_{target}$
olduğunu doğrula.

T-S02 — Velocity boundary

Quintic trajectory için:

dq(0) ≈ 0

dq(T) ≈ 0

olduğunu doğrula.

T-S03 — Acceleration boundary

ddq(0) ≈ 0

ddq(T) ≈ 0

olduğunu doğrula.

T-S04 — Joint limits

Trajectory'nin hiçbir örneğinde joint limit ihlali olmamalı.

T-S05 — Velocity limits

Velocity limitleri aşılmamalı.

T-S06 — Acceleration limits

Acceleration limitleri aşılmamalı.

T-S07 — Determinism

Aynı girişler aynı trajectory çıktısını üretmeli.

T-S08 — Multi-waypoint

Test:

current_q → q_A → q_B

Trajectory sırasıyla iki waypoint'e ulaşmalı.

T-S09 — FK final-pose validation

Son q değeri FK'ya sokulduğunda beklenen Cartesian hedefe ulaşmalı.

T-S10 — Existing regression

Trajectory modülü eklendikten sonra mevcut:
Foundations
Core
Hybrid
testlerinin tamamı çalışmaya devam etmeli.

---

14. Demonstration Scenario

KUKA KR6 R900 sixx için deterministik bir demonstration oluştur.
Başlangıç:
q_current

Ardından iki erişilebilir Cartesian pose tanımla:

Pose A

Pose B

Akış:

Pose A ↓ IK ↓ q_A

Pose B ↓ IK ↓ q_B

ardından:

q_current → q_A → q_B

trajectory oluştur.

Çıktıda aşağıdakileri raporla:

- total duration
- timestep
- number of samples
- maximum joint velocity
- maximum joint acceleration
- joint-limit violations
- A noktası final position error
- B noktası final position error
- orientation errors

- kullanılan IK solver
- kullanılan interpolation yöntemi

Trajectory verisini dosyaya kaydet.

---

15. Görseller

Test/demo çıktılarından en az şu grafikler üret:

1. Joint position vs time
2. Joint velocity vs time
3. Joint acceleration vs time
4. End-effector XYZ trajectory
5. Cartesian trajectory'nin 3B görünümü

Grafikler raporlama/artifact klasörüne kaydedilmelidir.

---

16. Kabul Kriterleri

Aşama yalnızca aşağıdaki koşullar sağlandığında PASS kabul edilmeli:

- current_q → q_target trajectory oluşturulabiliyor.
- A → B multi-waypoint hareketi oluşturulabiliyor.
- Quintic interpolation çalışıyor.
- Joint position limitleri korunuyor.
- Velocity limitleri korunuyor.
- Acceleration limitleri korunuyor.
- Başlangıç ve bitiş velocity/acceleration koşulları doğrulanıyor.
- FK son hedefi doğruluyor.
- Deterministic execution sağlanıyor.
- Mevcut test suite regression vermiyor.
- Demo artifactları üretiliyor.

Herhangi bir safety/limit kontrolü başarısızsa PASS verme.

---

17. Bu Aşamada Yapılmaması Gerekenler

Scope creep oluşturma.
Aşağıdakileri uygulama:

- ROS2 fiziksel motor kontrolü
- Gazebo entegrasyonu
- MoveIt entegrasyonu
- obstacle avoidance
- collision-free global planning
- RL motion planning
- dynamics / torque optimization
- visual servoing

Ancak mimari bunların daha sonra eklenmesini engellemeyecek şekilde tasarlanmalıdır.

---

18. Dokümantasyon

Roadmap ve README'yi güncelle.
NeuroKinematics mimarisi açıkça şu şekilde gösterilmelidir:
Target Pose ↓ IK Solver ↓ Target Joint Configuration ↓ Trajectory Generator ↓ Trajectory Validation ↓ Simulation / Studio ↓ Future ROS2 / Physical Robot
Trajectory Generation ile Motion Planning arasındaki fark dokümantasyonda açıkça belirtilmelidir.
Trajectory Generation:
“Bilinen başlangıç ve hedef joint

konfigürasyonları arasında uygulanabilir zaman-parametreli hareket üretimi.”
Motion Planning:
“Engeller ve çevresel kısıtlar altında geçerli yol arama.”
Bu aşamanın yalnızca birincisini gerçekleştirdiği açıkça yazılmalıdır.

---

19. Git

Çalışmaya başlamadan önce:
git status
git branch
git log -1
kontrol et.
İlgisiz dosyaları değiştirme.
Uygulamayı mantıksal commitlere ayır.

Öneri:

1. trajectory data model + interpolation
2. limits + timing
3. waypoint + IK integration
4. validation + tests
5. demo + documentation

Son durumda:
git status
test sonuçları
commit SHA
değiştirilen dosyalar
üretilen artifactlar
raporlanmalıdır.

Nihai Çıktı Formatı

Çalışma tamamlandığında şu yapıda rapor ver:

Status

PASS / PARTIAL / FAIL

Architecture

Implemented Features

Tests

Test	Result	Metric

KUKA Demonstration

current → A → B

Trajectory Metrics

Generated Artifacts

Regression Results

Known Limitations

Git

Next Recommended Step

Başarısız veya eksik testleri gizleme.
PASS kararı yalnızca kabul kriterlerinin tamamı sağlanıyorsa verilmelidir.